

SAMSUNG PRISM

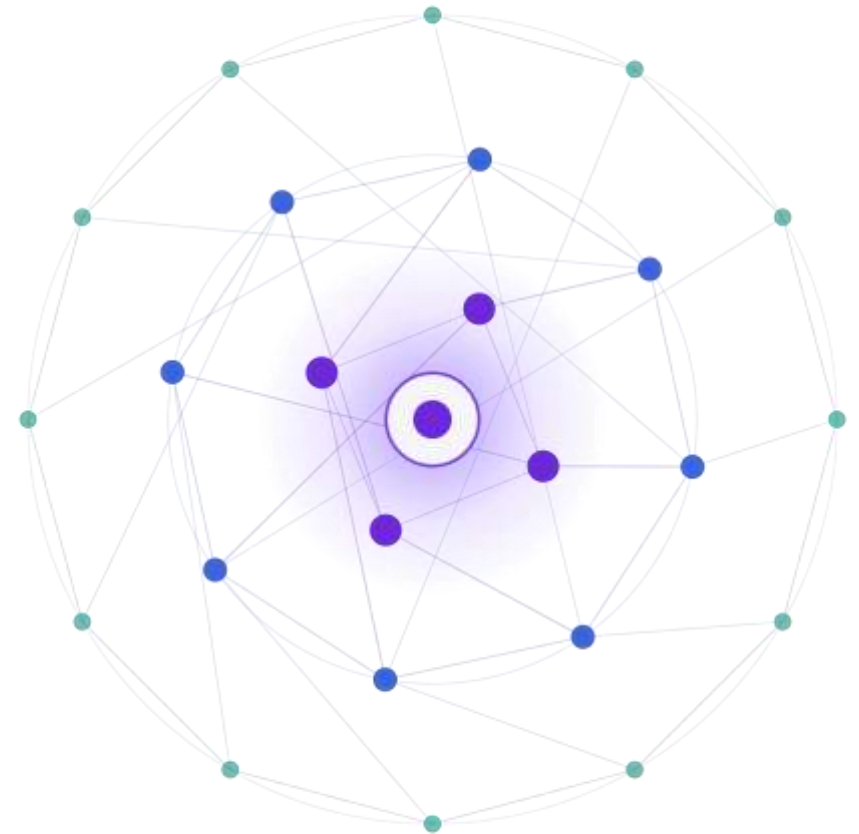
Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - 04 · Streaming Live RAG
- Team Name - Coding Agent RIT
- College Name - M S Ramaiah Institute of Technology, Bengaluru
- Member Name & Email 1- Cheshta Rajput (Presenter)
- Member Name & Email 2-
- Member Name & Email 3-
- Member Name & Email 4-
- Submission Github link - github.com/cheshtarajput098-arch/kairos

KAIROS Streaming Live RAG that answers while you speak

Speculative retrieval · multi-intent decomposition · in-place state evolution · 100% offline on a laptop CPU



Theme

Voice assistants should answer while you are still speaking, not seconds after you stop

THEME 04

Streaming Live RAG

Build a retrieval-augmented voice assistant that works on a live speech stream: retrieve and ground answers during the utterance, handle compound questions, and adapt when the speaker changes their mind.

Our entry: KAIROS, an event-driven streaming RAG engine that runs fully offline on CPU.

1

Answer while you speak

Speculatively retrieve and draft verified answers from speech prefixes, before the speaker stops talking.

2

Understand compound questions

Split one utterance into parallel intents instead of one noisy vector search.

3

Evolve, don't restart

Patch only the claim a late correction affects; keep every other claim byte-identical.

4

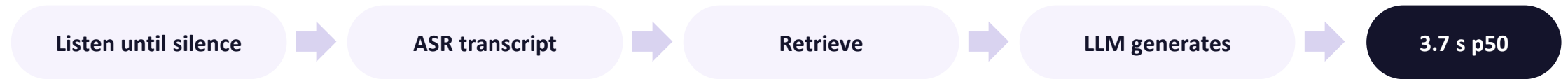
Stay grounded and private

Every claim maps to a verbatim source span; the whole pipeline runs locally with zero network.

Existing Solutions & Gaps

Conventional voice RAG is sequential: nothing starts until the speaker has finished

HOW EXISTING VOICE ASSISTANTS WORK



3.5 – 5.0 s

Awkward silence

Sequential batch processing (listen → ASR → retrieve → LLM) leaves users waiting in complete silence after they speak.

Fails

On compound questions

“I need a venue in Pune for 40 people, and what's the refund policy?” is sent as one noisy vector search.

100% waste

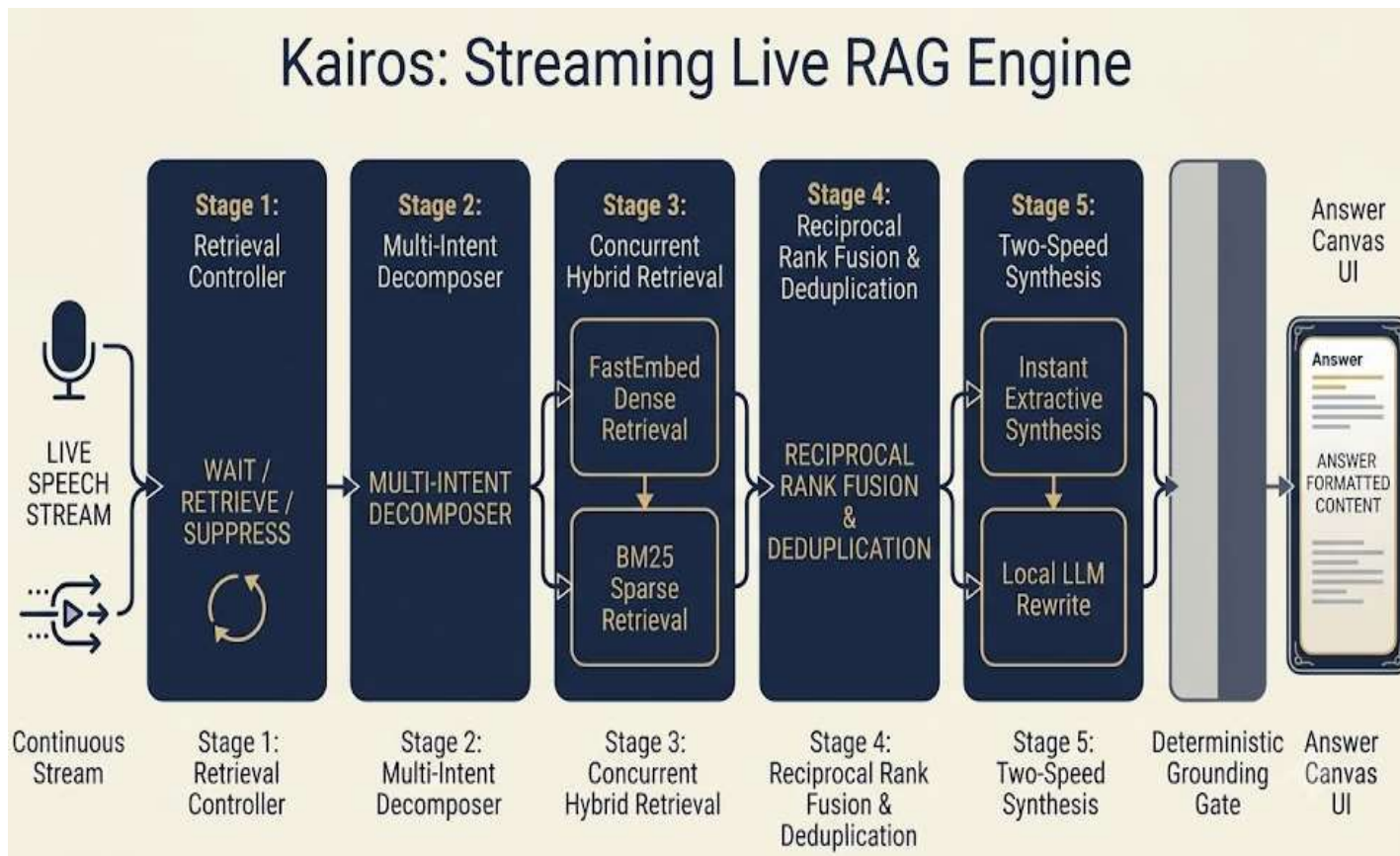
On detail revisions

Late self-repairs (“Actually, make that 60 people”) force naive engines to discard state and re-run the whole turn.

KAIROS: moves retrieval and answer drafting into the utterance window, so the answer is verified the moment speech stops.

Our Solutions & Architecture Diagram

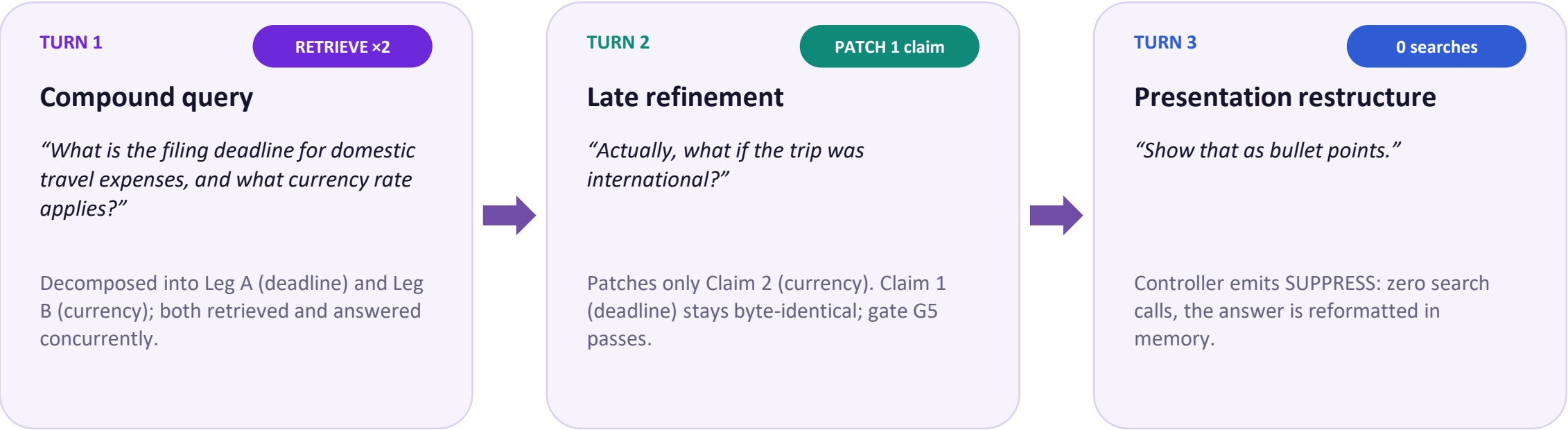
A parsimonious 5-stage pipeline on one asyncio event loop, with no LangChain or LlamaIndex



- 1 Streaming controller**
WAIT / RETRIEVE / SUPPRESS per speech chunk in under 1 ms
- 2 Multi-intent decomposer**
Strips disfluencies, splits into ≤ 4 parallel legs
- 3 Hybrid retrieval**
BGE-small dense + BM25 sparse, 400 ms deadline
- 4 Rank fusion**
Reciprocal Rank Fusion ($k = 60$) + dedup
- 5 Two-speed synthesis**
< 1 ms extractive, then < 820 ms grounded rewrite behind gate G4

Demo & Product Walkthrough

One conversation, three turns: decompose, evolve in place, restructure without searching



PRODUCT WALKTHROUGH · CALM PRECISION UI

Board 6: Ask / Home
Mic orb + 3 guided scenario cards for one-click replay

Board 7: Live Conversation
Colour-threaded intents, verified-sentence counter, quote drawer

Board 8: Knowledge Explorer
Chunk inspector with BGE + BM25 scores, SHA-256 manifest

Board 9: Telemetry & Traces
Live Gantt of speech chunks, decisions and retrieval legs

Tools and tech stack used

Everything runs locally: open models, no cloud APIs, no agent frameworks

1

Speech → text stream

Chunked streaming input with prefix-hash caching per chunk

2

Dense retrieval

FastEmbed · BGE-small embeddings

3

Sparse retrieval

BM25s index, fused via Reciprocal Rank Fusion (k = 60)

4

Local LLM

Qwen2.5-1.5B-Instruct
Q4_K_M GGUF (Apache 2.0) via llama-cpp-python

5

Orchestration

Python asyncio, single event loop; no LangChain, no LlamaIndex

6

Observability

OpenTelemetry spans with microsecond per-stage timings

7

Packaging & eval

Docker Compose offline container; make eval runs gates G1–G6

8

Security & UI

STRIDE, OWASP GenAI Top 10, spotlighting, PII redaction · WCAG 2.2 AA UI

Target hardware: laptop-class CPU, 2.0 vCPU and a 4 GB RAM budget

Impact & Use case

Natural, zero-pause conversation over private documents, on hardware people already own

1.508 s

saved per turn (median)

vs the sequential baseline; 100% of retrieval turns saved time

0 / 112

hallucinated claims

every claim maps byte-for-byte to a source span (gate G4)

584 MB

RAM for one session

fully offline on CPU; 10 concurrent sessions fit in 1.5 GB

WHERE IT FITS

- 1

Enterprise policy assistant
Employees ask about travel, expense and HR rules and get cited answers mid-sentence (demo scenario 2).
- 2

Booking and service concierge
Compound requests like venue + headcount + refund policy, with corrections handled in place.
- 3

Privacy-sensitive on-device voice
No network, PII masked at the stream boundary: suits phones, cars and regulated workplaces.
- 4

Contact-centre agent assist
Grounded snippets surface while the caller is still talking, with sources shown for trust.

Innovation highlights, results and limitations

All six acceptance gates pass; here is what works, and what does not yet

Gate	Metric	Threshold	Measured (n)	Status
G1	Offline reproducibility	1.0	1.0 (zero network)	PASS
G2	Early retrieval triggering	≥ 0.80	1.000 (n=52)	PASS
G3	Multi-intent decomposition	≥ 0.70	0.895 (n=19)	PASS
G4	Grounding (hallucinations)	≤ 0.00	0.0000 (n=112)	PASS
G5	Selective state continuity	1.0	1.000 (n=18)	PASS
G6	Telemetry schema	1.0	1.000 (n=64)	PASS

Innovation highlights

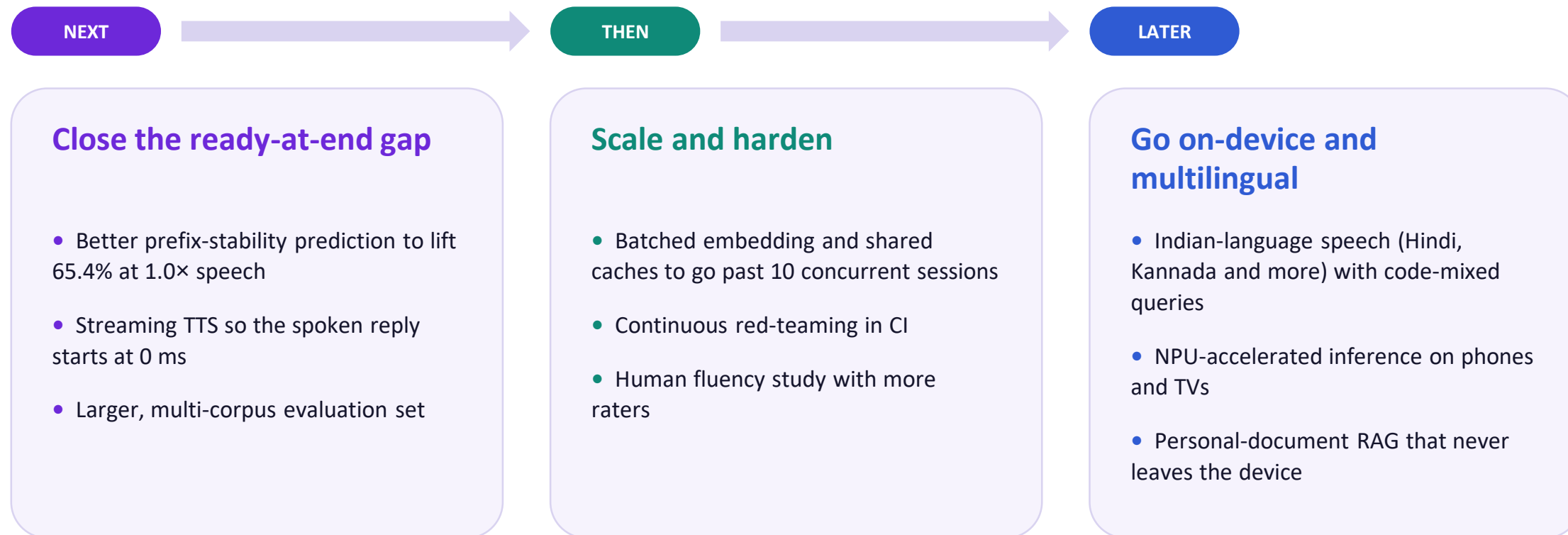
Ready-at-end drafting (65.4% of turns verified the moment speech ends) · two-speed grounded synthesis · in-place delta evolution of claims

Limitations (honest view)

- At natural 1.0× speech rate, 18 of 52 retrieval turns (34.6%) are not yet fully drafted when speech ends.
- At 10 concurrent sessions p50 latency rises to 8.5 s; at 25 the engine load-sheds.
- A 1.5B local model limits multi-step reasoning depth in rewrites.
- Fluency study is small (n = 20 turns, $\kappa = 0.689$); eval uses a single frozen corpus.

What's next

Planned next steps, building on the limitations we measured



Brownie points slide(differentiation)

Metrics that set KAIROS apart from turn-taking voice RAG

65.4%

Ready at end of speech

34 / 52 turns verified the exact millisecond speech ended; 100% at 0.75× and 1.5×

1,800 ms

Median lead time

Answers retrieved and verified 1.8 s before the speaker goes silent

0.0 ms

Time to first token

p50 TTFT, because the answer is pre-drafted in memory

0.0%

Attack success rate

0 / 32 red-team injection turns succeeded under corpus spotlighting

5.0 / 5

Blind fluency score

Grounded rewrite vs 3.92 for raw quotes (+1.08), 0% hallucination

0

Agent frameworks

One asyncio loop and 5 stages; 0.01 ms security overhead per turn

Checklist- Updated on Public GitHub

Release tag PRISM_GENAI_HACKATHON_Y2026 on github.com/cheshtarajput098-arch/kairos



Working prototype code: public or shared GitHub repo

github.com/cheshtarajput098-arch/kairos



README with reproducible setup instructions

docker compose up brings the system up; make eval re-runs gates G1–G6 offline



Demo video, max 5 minutes (YouTube or Drive link)

Link: add before submitting



Presentation file (PPT or PDF)

This deck, plus architecture brief, telemetry schema, runbook and STRIDE threat model

Thank you

KAIROS · Team Coding Agent RIT · M S Ramaiah Institute of Technology

Presenter: Cheshta Rajput · github.com/cheshtarajput098-arch/kairos · We welcome your questions

Submitted on: 29 September 2026

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India